CHARGEASSERT

MVP PRODUCT + TECHNICAL OVERVIEW

ChargeAssert

Financial release testing for EV charging.

A vendor-neutral test harness that catches missing, duplicate and incorrectly priced charging records **before** a software release reaches production.

OWNER
Selim Abouleila

STATUS
Implementation blueprint

VERSION
0.1 / August 2026

Independent educational and portfolio project. Public and synthetic data only.

01 / CHARGEASSERT

The product in one page

ChargeAssert turns financial correctness into a software-release requirement. It replays the same EV-charging journeys through a baseline and a candidate, then blocks the release if the candidate creates a materially worse billing outcome.

SIMPLE EXPLANATION

ChargeAssert is a **financial crash-test laboratory** for EV-charging software.

TESTS
Realistic, deterministic charging lifecycles with late events, retries, tariff changes and controlled faults.

FINDS
Missing CDRs, semantic duplicates, wrong energy, stale tariffs, overbilling and operator leakage.

DECIDES
A reproducible PASS / FAIL gate with first divergence, affected sessions and modeled financial exposure.

What makes the MVP valuable

BUYER QUESTION	CHARGEASSERT ANSWER
Can this release go live safely?	Run the same scenario manifest against baseline and candidate, then apply explicit financial thresholds.
What exactly broke?	Show the first event, retry or tariff decision where candidate behavior diverged.
Who could be harmed?	Separate customer overbilling from operator underbilling instead of reporting one generic error count.
Can engineering reproduce it?	Persist the Git SHAs, run ID, seed, snapshot checksums, tariff hash and ordered evidence timeline.

PRIMARY USERS
QA and platform engineering, revenue assurance, finance operations, OCPP / OCPI integration teams and charging-platform vendors.

MVP OUTPUT
GitHub status, Delta evidence tables, Markdown / JSON summary, dashboard-ready Gold metrics and a reproduction command.

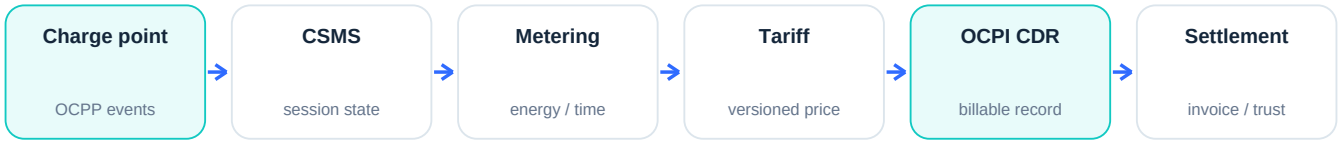
POSITIONING HYPOTHESIS

Most protocol tools focus on conformance and most reconciliation happens after production. ChargeAssert focuses on **pre-release financial outcomes**. This is a product hypothesis to validate, not a claim that no adjacent solution exists anywhere.

02 / CHARGEASSERT

Why a technically successful session can still lose money

A charging journey crosses protocol, state, metering, pricing and billing boundaries. Each system can report success while the final financial record is missing, duplicated or wrong.



Technical success can still hide a financial failure anywhere after the first event.

FAILURE	CUSTOMER / OPERATOR EFFECT	PRE-RELEASE ASSERTION
Completed session, no CDR	Operator revenue leakage	Exactly one final CDR within the project SLA
Retry creates a second CDR	Customer overbilling, refunds, trust damage	No second non-credit CDR for one logical session
Wrong meter delta or unit	Overbilling or underbilling	CDR energy matches validated meter evidence
Latest tariff applied retroactively	Wrong price and dispute risk	As-of tariff version matches each charging period
Out-of-order / late event	Wrong duration or premature close	Event-time state is correct and business latency is explicit

CUSTOMER
Prevent duplicate and inflated charges before they reach a statement.

OPERATOR
Protect billable revenue and expose the exact missing or understated record.

ENGINEERING
Replace long investigations with a deterministic failure timeline and reproduction command.

LEADERSHIP
Make release risk visible in operational and modeled financial terms.

MODELED MONTHLY EXPOSURE — **candidate defect-rate delta x assumed monthly sessions x assumed impact per affected session**

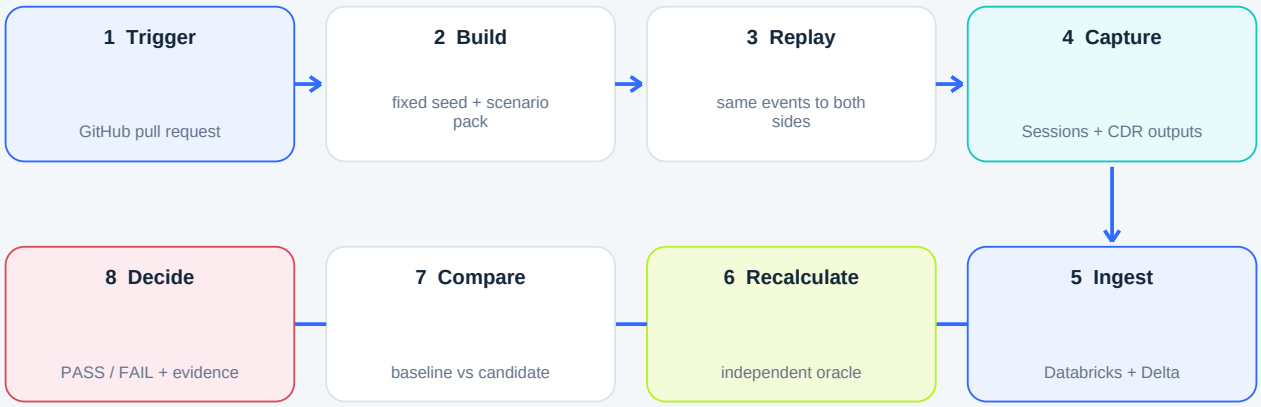
Every projection must display its assumptions. It is modeled exposure - never actual loss, recovered revenue or proven savings.

03 / CHARGEASSERT

How ChargeAssert works

The portfolio MVP is fully controlled and reproducible. A future adopter can replace the mock adapter with approved staging endpoints without giving ChargeAssert access to proprietary source code.

ONE DETERMINISTIC RUN - TWO RELEASES - ONE FINANCIAL VERDICT



BLACK-BOX CONTRACT

The system under test only needs to accept versioned charging inputs and expose resulting Sessions and CDRs. Source-code access is not required.

INDEPENDENT ORACLE

Expected energy, tariff version and amount are calculated before candidate output is read. The baseline helps comparison but is not treated as unquestionable truth.

GitHub and Databricks have different jobs

GITHUB - CONTROL PLANE	DATABRICKS - EXECUTION PLANE
Source code, scenario definitions, schemas, tariff fixtures, gate thresholds, review history and the final pull-request status.	Distributed generation and replay, streaming ingestion, Delta transformations, independent pricing, comparison, evidence and metrics.

KEY REPRODUCIBILITY RULE

One run manifest pins the scenario seed, synthetic clock, input snapshot checksums, tariff hash, baseline SHA, candidate SHA and runtime version. The same manifest should produce the same verdict.

04 / CHARGEASSERT

A credible MVP - deliberately narrow

The strongest first version proves one complete financial release gate. It does not pretend to be a production payment platform, full protocol validator or universal tariff engine.

INCLUDED IN MVP

- Deterministic synthetic session generation
- Pinned public-data-derived behavior and station profiles
- Documented OCPP 2.0.1 Edition 3 event subset
- OCPI-aligned Sessions, Tariffs and CDR subset
- Known-good baseline and deliberately faulty candidate
- Configurable fault injection with hidden labels
- Independent EUR pricing oracle with Decimal arithmetic
- Bronze / Silver / Gold Delta pipeline
- GitHub release gate plus evidence package
- At least eight reproducible fault scenarios

EXPLICITLY OUTSIDE MVP

- Real card authorization or payment capture
- Bank, PSP, ERP, invoice or clearing settlement
- Full OCPP / OCPI certification or conformance
- Unapproved access to any operator environment
- Every tariff, tax system, currency or country
- Production monitoring and automated recovery
- Multi-tenant commercial SaaS capabilities
- Machine learning in the first release
- Claims of proven real-world savings
- Confidential employer or customer data

MVP definition of done

01
The reference passes; every injected fault makes the intended assertion fail without leaking its label to the assertion engine.

02
A defective candidate fails the GitHub check; the demonstrated fix passes the exact same manifest.

03
The report identifies affected sessions, expected vs actual values, first divergence and modeled impact.

04
Replays are idempotent, provenance is complete and another developer can deploy from the documentation.

05 / CHARGEASSERT

Fault catalogue and expected business behavior

Scenarios contain inputs and expected outcomes - not just pre-labelled anomaly rows. Fault labels stay outside the assertion engine and are used afterward to measure detection coverage.

ID	SCENARIO	CONTROLLED FAILURE	FINANCIAL ASSERTION
S01	Happy path	Valid lifecycle	Exactly one correct final CDR
S02	Missing end	Transaction end never arrives	No false bill; incomplete state is explicit
S03	Transport duplicate	Same event_id delivered twice	Duplicate input is idempotently ignored
S04	Semantic duplicate	Two CDR IDs for one session	Second non-credit CDR fails the gate
S05	Retry after HTTP 500	Receiver processed request but response failed	Retry must not create a second charge
S06	Out-of-order meter	Meter update arrives late	Event time produces the correct energy
S07	Late transaction end	Final event exceeds project SLA	Correct result plus explicit latency failure
S08	Tariff boundary	Rate changes during session	Historical as-of tariff is selected
S09	Meter reset / units	Register drops or Wh becomes kWh	Impossible energy is quarantined
S10	Clock boundary	Timezone / daylight-saving transition	UTC event-time duration stays correct
S11	Schema evolution	New or malformed field	Rescue safely or fail the contract
S12	Credit correction	Original needs correction	Credit + replacement; no mutation or double count

IMPORTANT DISTINCTION

A **transport duplicate** repeats the same immutable event ID and should be deduplicated. A **business duplicate** creates two different CDR IDs for one session and must be preserved as evidence and reported as a defect.

PULL REQUEST PACK

Fast deterministic smoke suite. Initial target: 10,000 sessions, subject to measured cost and runtime.

SCHEDULED PACK

Larger benchmark and schema-drift suite. Initial target: 100,000 sessions, explicitly a validation target rather than a performance claim.

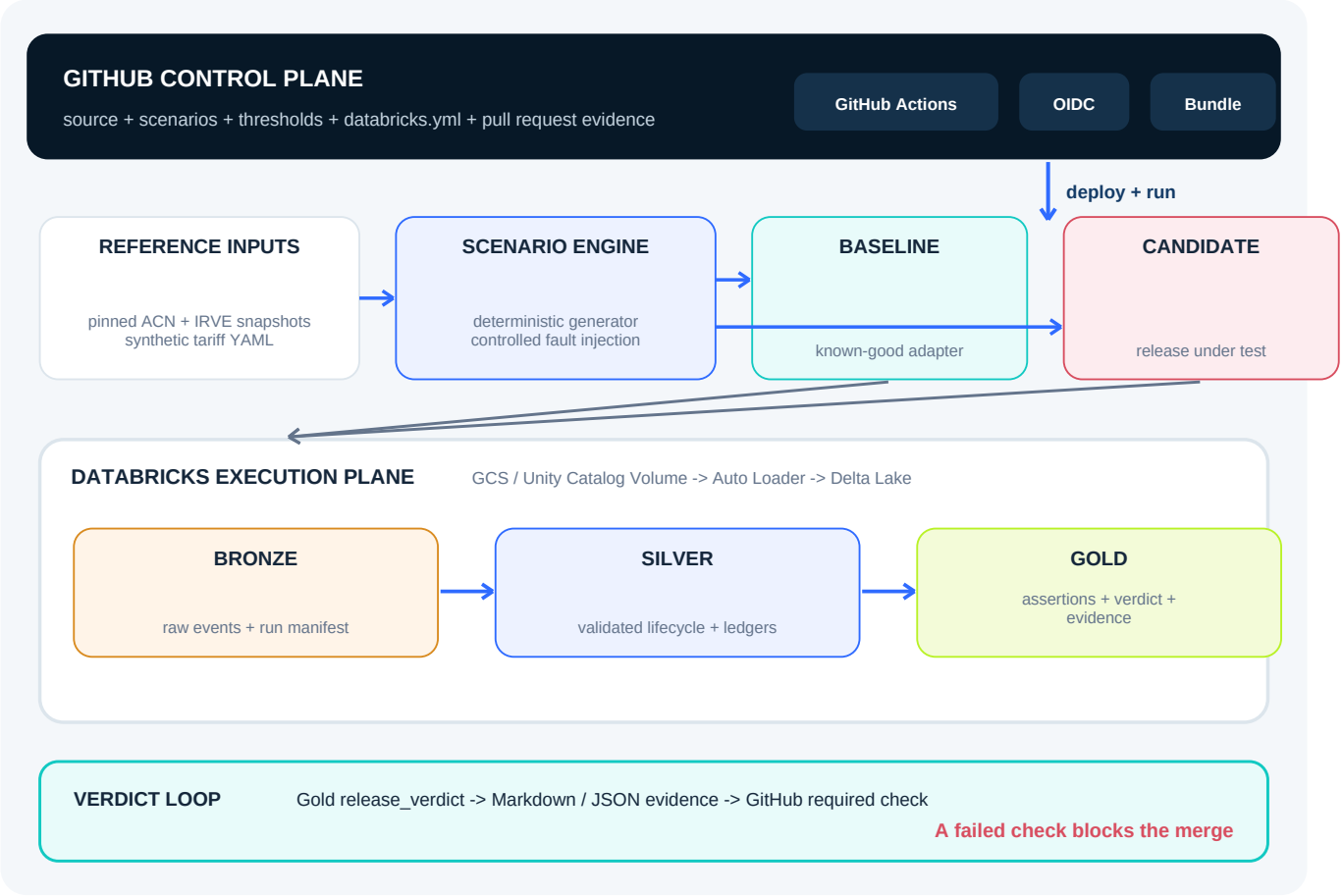
GOLDEN CASES

Small manually reviewed fixtures for pricing, rounding, tariff boundaries, credits and known expected CDRs.

06 / CHARGEASSERT

Databricks and GCP architecture

GitHub controls the release; Databricks executes the financial test. The pipeline uses immutable evidence, explicit event-time handling and a business-aware distinction between technical and semantic duplicates.



STREAMING

Auto Loader ingests immutable JSON micro-batches. AvailableNow processes the test set and terminates. Checkpoints do not replace explicit source deduplication.

EVENT TIME

Watermarks bound state and handle lateness. A business CDR SLA is evaluated separately from Spark's technical watermark.

CI / CD

Declarative Automation Bundles define jobs and resources. GitHub Actions should use workload identity federation / OIDC where available.

GOVERNANCE

Unity Catalog separates CI and development, controls evidence access and preserves lineage, provenance and discoverability.

Recommended catalog pattern: **chargeassert_dev** and **chargeassert_ci**, each with Bronze, Silver and Gold schemas. Production logic lives in testable Python modules; notebooks remain optional for exploration and demonstrations.

07 / CHARGEASSERT

Evidence model and financial assertions

Every verdict must be explainable at session level. Raw protocol-shaped evidence is retained, normalized into a canonical lifecycle, independently priced and compared before a release decision is written.

LAYER	CORE TABLES	PURPOSE
Bronze	run_manifest, ocpp_events_raw, ocpi_sessions_raw, ocpi_cdrs_raw, tariffs_raw	Immutable payload, source file, event / ingest time, hashes, Git SHAs and rescued data
Silver	transaction_event, session_lifecycle, tariff_history, cdr, expected_ledger, actual_ledger, quarantine	Validated business objects, SCD2 tariffs, event-time state and independent expected outcome
Gold	assertion_result, release_comparison, release_verdict, failure_evidence, scenario_metrics	Explainable rule failures, financial deltas, first divergence and final gate

Core invariant set

ID	RULE	ASSERTION
A01	Completeness	Every completed billable session produces exactly one normal final CDR within a configured project SLA.
A02	No double billing	Different CDR IDs must not bill the same logical session twice.
A03	Energy	CDR energy matches the validated meter-register difference within a declared tolerance.
A04	Price	CDR total matches the independent tariff calculation within currency rounding tolerance.
A05	Tariff time	The tariff version is valid for each relevant charging period; no retroactive latest-rate lookup.
A06	Correction	Historical CDR evidence is not mutated; correction uses the supported credit / replacement flow.

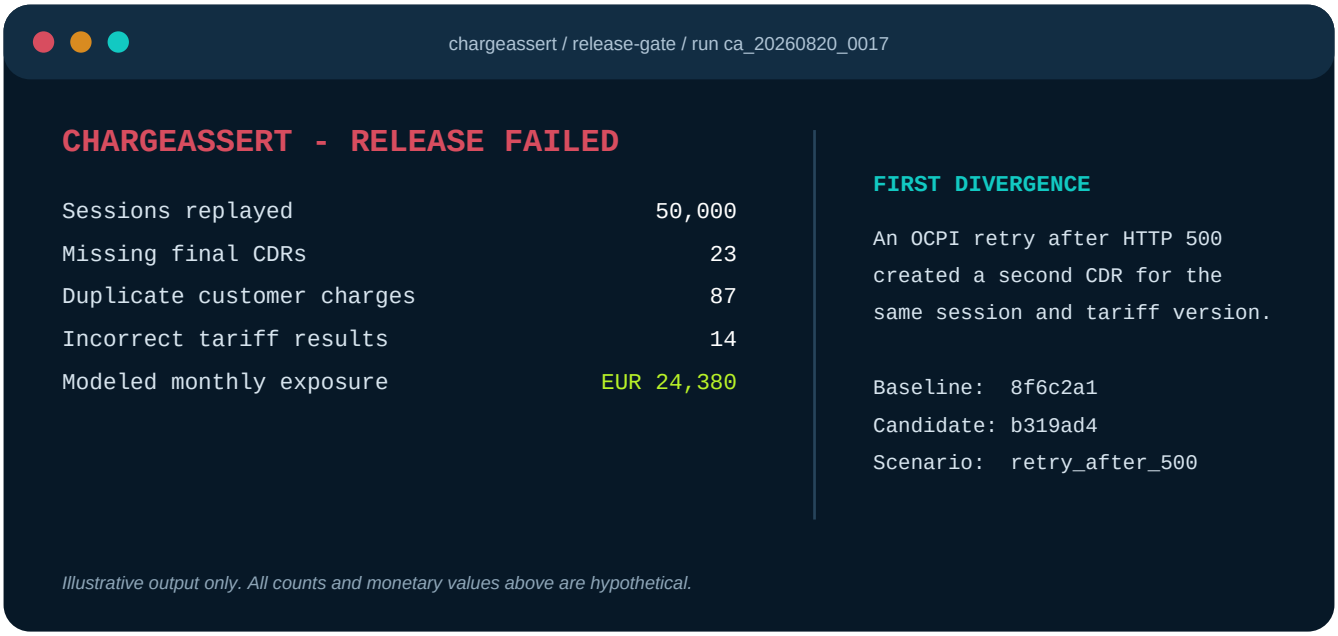
ORACLE FORMULA

Expected energy (kWh) = (end meter Wh - start meter Wh) / 1,000. Expected total = fixed + energy + time + parking components + explicit tax treatment. All amounts use Decimal arithmetic and documented rounding.

08 / CHARGEASSERT

The release gate is the product moment

The strongest demonstration is not a dashboard. It is a pull request that fails for a financial reason, explains the first divergence, and turns green after the exact defect is fixed.



GATE TYPE	EXAMPLE POLICY	WHY
Hard fail	customer_overcharge_eur > 0; semantic_duplicate_cdrs > 0	Protect customers and preserve idempotency
Relative fail	missing_cdr_rate delta > 0.05 percentage points	Prevent candidate regression without hiding baseline debt
Latency fail	p95 CDR latency increase > 30 seconds	Make operational degradation visible
Evidence fail	missing seed, SHA, tariff hash or event timeline	No release verdict without reproducibility

PASS / FAIL
Machine-readable verdict and non-zero task result.

FIRST DIVERGENCE
Earliest event, retry or pricing decision that explains the change.

WHO IS EXPOSED
Customer overbilling and operator underbilling reported separately.

REPRODUCE
Run ID, manifest and one command to replay the exact case.

09 / CHARGEASSERT

APIs, protocol contracts and data provenance

Public APIs make the simulation realistic; they are not the software being tested. The MVP caches versioned snapshots so every pull request remains deterministic and respectful of source terms.

OCPP SIDE

Synthetic OCPP 2.0.1 Edition 3-shaped transaction events represent station-to-CSMS behavior. Scope is versioned and narrow; ChargeAssert is not an OCPP-certified implementation.

OCPI SIDE

An OCPI-aligned Sessions, Tariffs and CDR subset represents system outputs. Pin the exact official branch / tag and never claim full OCPI conformance.

FUTURE STAGING

A consenting customer supplies WebSocket / REST endpoints, credentials and tariff rules. ChargeAssert evaluates black-box inputs and outputs - not arbitrary companies on the internet.

SOURCE / API	MVP USE	GUARDRAIL
ACN-Data REST API /api/v1/sessions/{site_id}	Energy, connection and duration patterns	Token required; educational / research terms; cache a bounded snapshot; avoid /ts in routine CI
French IRVE dataset API /api/datasets/5448d3e0c751df01f85d0572	Static French EVSE location, connector and power profiles	Metadata only - no real sessions, CDRs or machine-readable operator tariffs; pin resource and checksum
Scenario + tariff YAML	Synthetic IDs, event clock, rates, SLA and gate thresholds	All monetary rules are illustrative, version controlled and labelled synthetic
SystemUnderTestAdapter	consume events; export Sessions and CDRs	Local mocks in MVP; approved staging integration later; secrets never committed

REQUIRED DISCLOSURE

All monetary outcomes are modeled. ACN-derived charging behavior is mapped synthetically to public French IRVE infrastructure metadata; no claim is made that an ACN session occurred at an IRVE station or used an actual operator tariff.

The public repository contains no employer records, customer data, private schemas, production tariffs, credentials, personal identifiers or payment-card information. Every generated record carries **synthetic = true** and a provenance reference.

10 / CHARGEASSERT

Six-week build plan and repository shape

Weeks 1-6 deliver the complete portfolio MVP. Two optional hardening weeks can add a staging adapter, larger benchmarks, stronger identity controls and a recorded demonstration.

WEEK	DELIVERABLE	VISIBLE PROOF
01	Foundation: package, databricks.yml, CI smoke test, schemas and provenance model	Bundle validates and a small job runs from GitHub
02	Inputs: pinned snapshots, canonical contracts, deterministic generator, happy path	Same seed creates the same manifest and golden CDR
03	Bronze / Silver: Auto Loader, schema rescue, event time, dedup and lifecycle	Late / duplicate scenario produces correct conformed state
04	Oracle: SCD2 tariff history, Decimal pricing, expected and actual ledgers	Boundary and rounding golden tests pass
05	Assertions: fault catalogue, evidence rows, comparison and modeled exposure	Deliberately faulty candidate fails every intended rule
06	Gate: GitHub status, report, dashboard, documentation and five-minute demo	Failure turns green after the exact idempotency fix

RECOMMENDED REPOSITORY

```
ChargeAssert/
|-- docs/OVERVIEW.pdf
|-- databricks.yml
|-- src/chargeassert/
|   |-- simulator/
|   |-- sut/
|   |-- pipelines/
|   |-- assertions/
|   `-- reporting/
|-- scenarios/
|-- schemas/
|-- fixtures/
|-- tests/
`-- .github/workflows/
```

TEST STRATEGY

- Unit: price, Decimal rounding, VAT, tariff boundary and fault injection
- Spark integration: watermarks, technical dedup vs business duplicates, replay and quarantine
- End to end: defective candidate fails; corrected candidate passes the same manifest
- Security: secret scanning, synthetic identifiers and least-privilege CI identity

The schedule is an implementation estimate, not a contractual commitment. Cost, workspace tier and runtime targets must be measured during the first end-to-end spike.

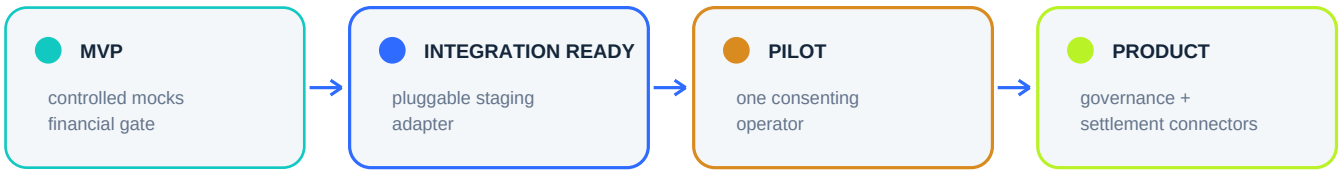
11 / CHARGEASSERT

Risks, roadmap and honest boundaries

ChargeAssert is strongest when it is precise about what the evidence proves. The MVP should earn trust through narrow contracts, reproducibility and explicit limitations.

RISK	MITIGATION
Synthetic scenarios miss production complexity	Use public behavior distributions, modular adapters, golden cases and clear coverage statements.
The oracle contains its own bug	Keep it independent from candidate logic; review golden calculations manually; test boundary and rounding cases.
Protocol scope expands uncontrollably	Pin a narrow OCPP / OCPI subset and publish an explicit compatibility matrix.
Modeled exposure is mistaken for real loss	Separate sample discrepancies from projections and show every volume / impact assumption.
Streaming makes results nondeterministic	Pin clocks, seeds, snapshots, IDs and runtime; use idempotent writes and versioned run manifests.
Public or confidential data is mishandled	Cache permitted snapshots, minimize fields, hash / drop user IDs, scan secrets and prohibit employer data.

Product roadmap



QUESTION	SHORT ANSWER
Does it process payments?	No. The MVP verifies Session and CDR financial correctness. PSP / bank failure handling is later scope.
Does it need source code?	No. A future user connects an approved staging adapter and supplies applicable test tariff rules.
Does it test arbitrary companies?	No. An operator or vendor must explicitly connect its staging environment.
Does it use machine learning?	No. Deterministic rules are more explainable with known expected outcomes. ML is optional only after labelled real data exists.

COMMERCIAL DIRECTION

After the MVP: validate with one consenting operator or platform vendor, measure false positives and investigation time, then add configurable staging adapters, contract packs and settlement evidence. Do not productize before validating the release-gate pain with real buyers.

12 / CHARGEASSERT

Reference set, glossary and project disclaimer

The MVP uses official protocol and Databricks documentation plus clearly bounded public data. Versions and snapshots must be pinned at implementation time because standards, schemas and product names evolve.

1. Databricks medallion architecture
2. Databricks Auto Loader for cloud object storage
3. Databricks watermarks and dropDuplicatesWithinWatermark
4. Lakeflow pipeline expectations
5. Declarative Automation Bundles
6. Databricks GitHub Actions and workload identity federation
7. Unity Catalog overview
8. Open Charge Alliance - OCPP protocols
9. Official OCPI specification repository
10. Caltech ACN-Data dataset and REST API
11. French national IRVE dataset
12. Official IRVE static schema documentation

Compact glossary

TERM	MEANING	TERM	MEANING
CPO	Charge Point Operator	eMSP	e-Mobility Service Provider
CSMS	Charging Station Management System	CDR	Charge Detail Record; billing-relevant session outcome
OCPP	Charge point to central-system protocol	OCPI	Roaming interface including Sessions, Tariffs and CDRs
Oracle	Independent expected financial calculation	SCD2	Effective-dated history that preserves tariff versions

PROJECT DISCLAIMER

ChargeAssert is an independent educational and portfolio project. It is not affiliated with or endorsed by any previous employer, charging operator, Open Charge Alliance, EV Roaming Foundation, Databricks or public-data provider. It is not a certification, legal, tax or accounting product.

THE MVP PROMISE

Catch missing, duplicate and incorrectly priced EV-charging records before the associated software release reaches production.